

PRESENTACIÓN

Árboles de Decisión

Modelos supervisados: CART y Random Forest

Profesor Ing. Pablo Sciolla

Universidad de San Andrés — Analítica de Datos

Mayo 2026

01

¿Qué vimos hasta acá? (1/2)

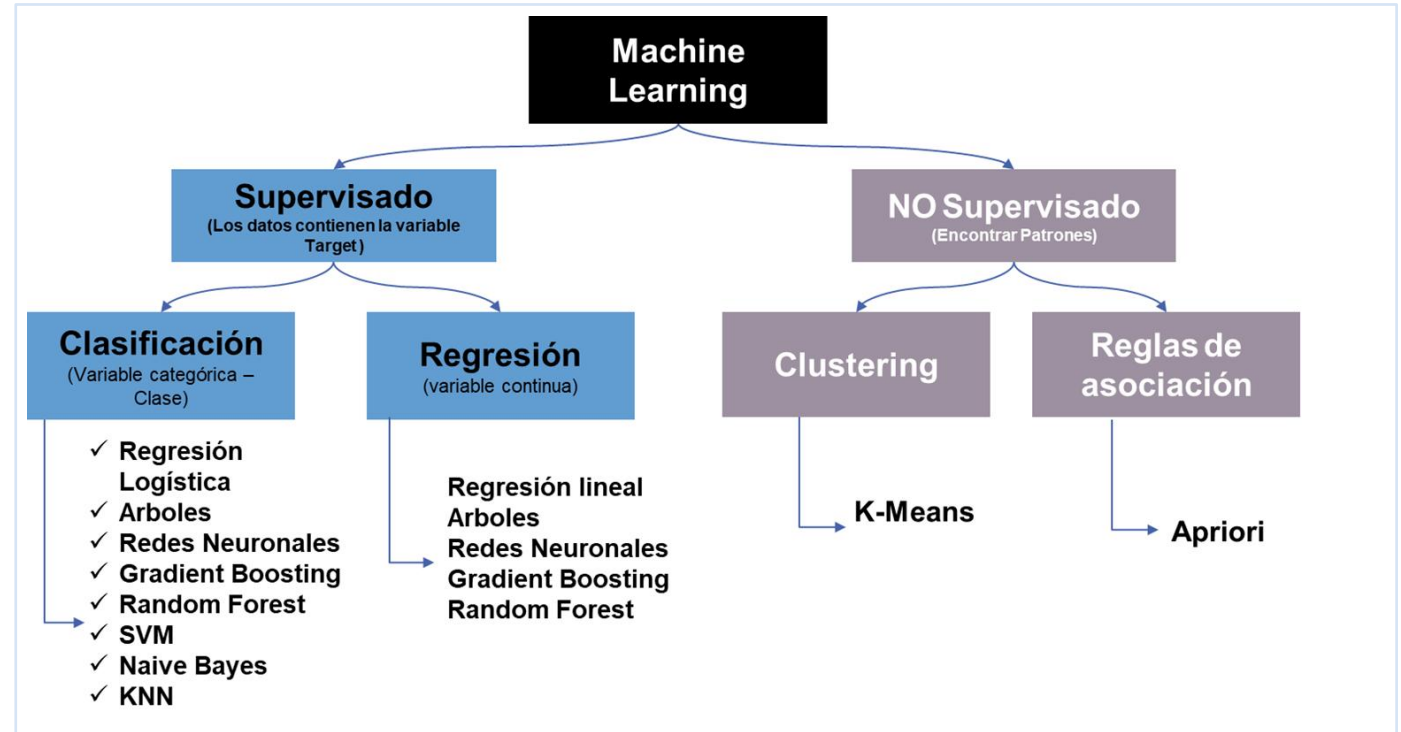
Recorrido de los temas abordados en las teóricas previas de aprendizaje supervisado.

- *Introducción al machine learning*
- *Diferencia entre métodos supervisados y no supervisados*
- *Problema a resolver y metodología de trabajo*
- *Métricas para medir la eficacia de un modelo*
- *Overfitting y underfitting*
- *KNN (K vecinos más cercanos)*
- *Algunos conceptos de regresión*
- *Librería de trabajo: scikit-learn (sklearn)*

¿Qué vimos hasta acá? (2/2)

Hacia dónde vamos

Continuamos con otros algoritmos para modelos supervisados: CART (árboles de clasificación y regresión) y Random Forest.



MARCO CONCEPTUAL

Métodos jerárquicos

Técnicas de clasificación basadas en agrupamiento iterativo

¿Qué son?

Son técnicas de agrupamiento de datos que, de forma iterativa, fusionan o dividen grupos armados en una etapa previa. Pueden ser aglomerativos, divisivos o mixtos.

Aglomerativos *Parten de cada observación por separado (cada observación es un grupo) y van fusionando grupos hasta obtener la configuración final.*

Divisivos *De forma contraria, parten de un único grupo que comprende todas las observaciones, el cual se va dividiendo hasta lograr los grupos definitivos.*

Mixtos *Comienzan como un método divisivo obteniendo más grupos de los necesarios y luego aplican un método aglomerativo para reagrupar y llegar al modelo final.*

PROBLEMA

Clasificación • dataset Titanic

Usaremos el dataset Titanic para predecir el destino de los pasajeros del barco. El dataset cuenta con las siguientes variables:

PassengerId	SibSp
Survived	Parch
Pclass	Ticket
Name	Fare
Sex	Cabin
Age	EAnálítica de Datosrked



Descripción del dataset

df.info()

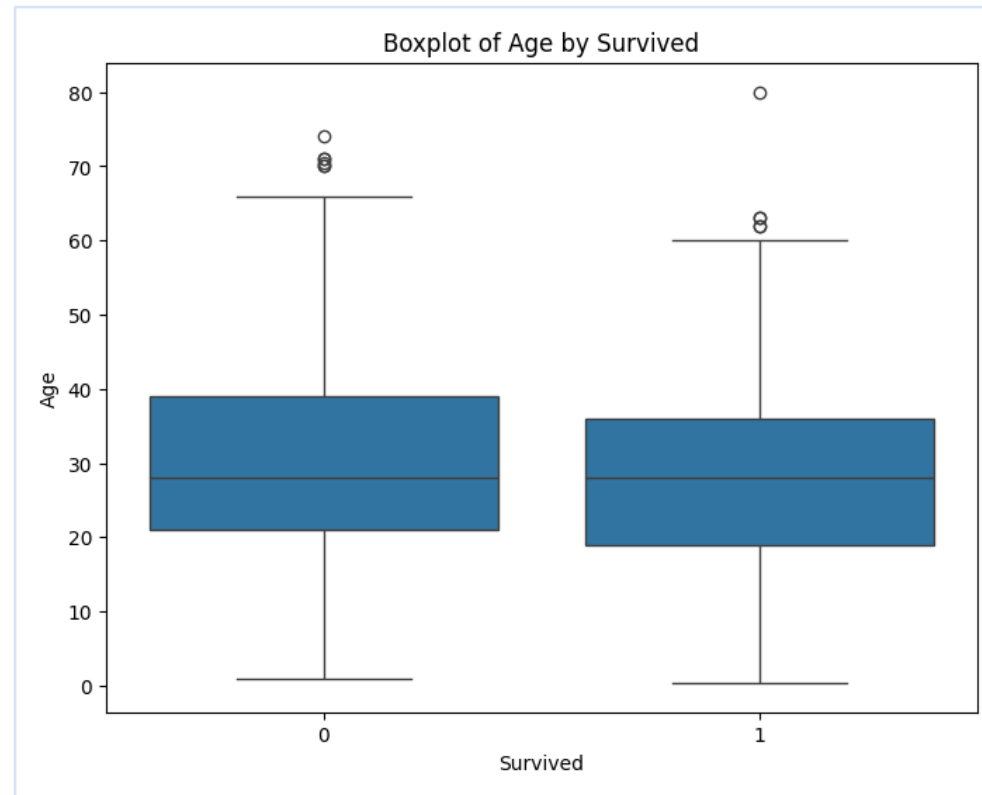
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
Column Non-Null Count Dtype

0 PassengerId 891 non-null int64
1 Survived 891 non-null int64
2 Pclass 891 non-null int64
3 Name 891 non-null object
4 Sex 891 non-null object
5 Age 714 non-null float64
6 SibSp 891 non-null int64
7 Parch 891 non-null int64
8 Ticket 891 non-null object
9 Fare 891 non-null float64
10 Cabin 204 non-null object
11 Embarked 889 non-null object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

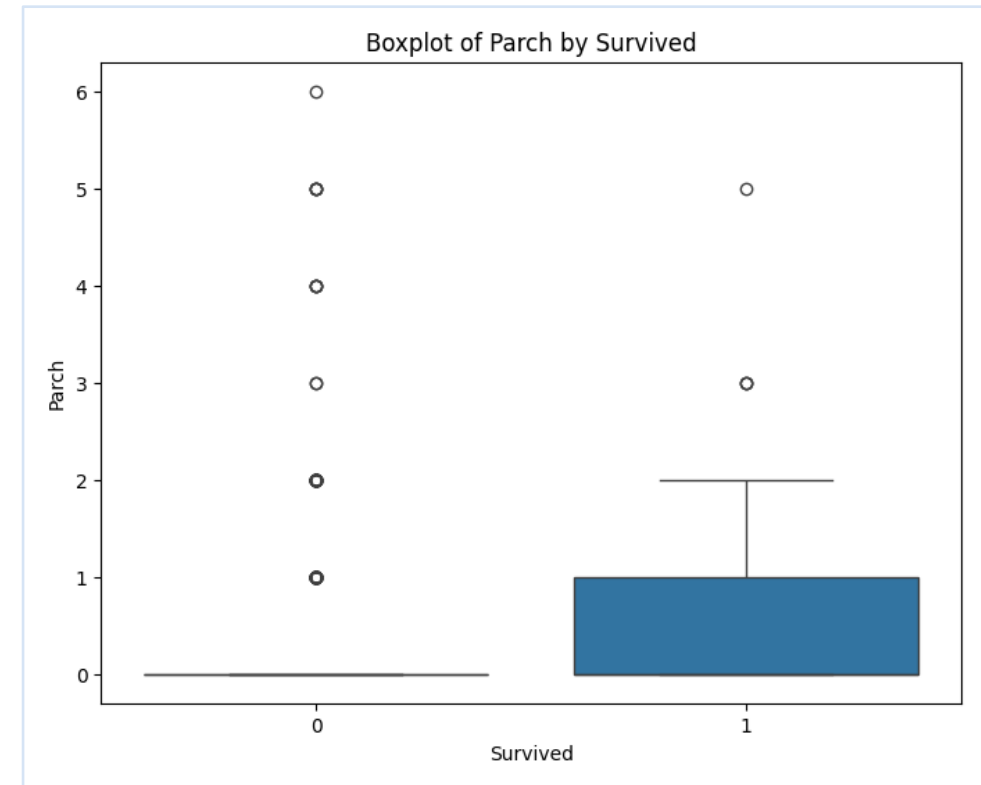
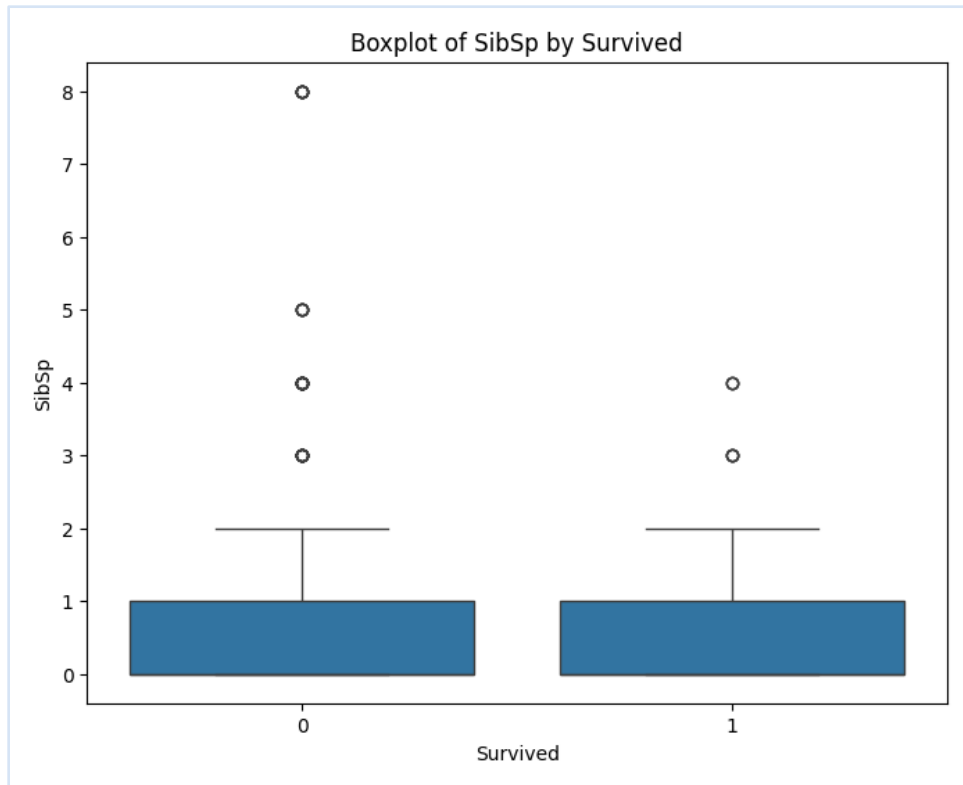
df.describe()

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

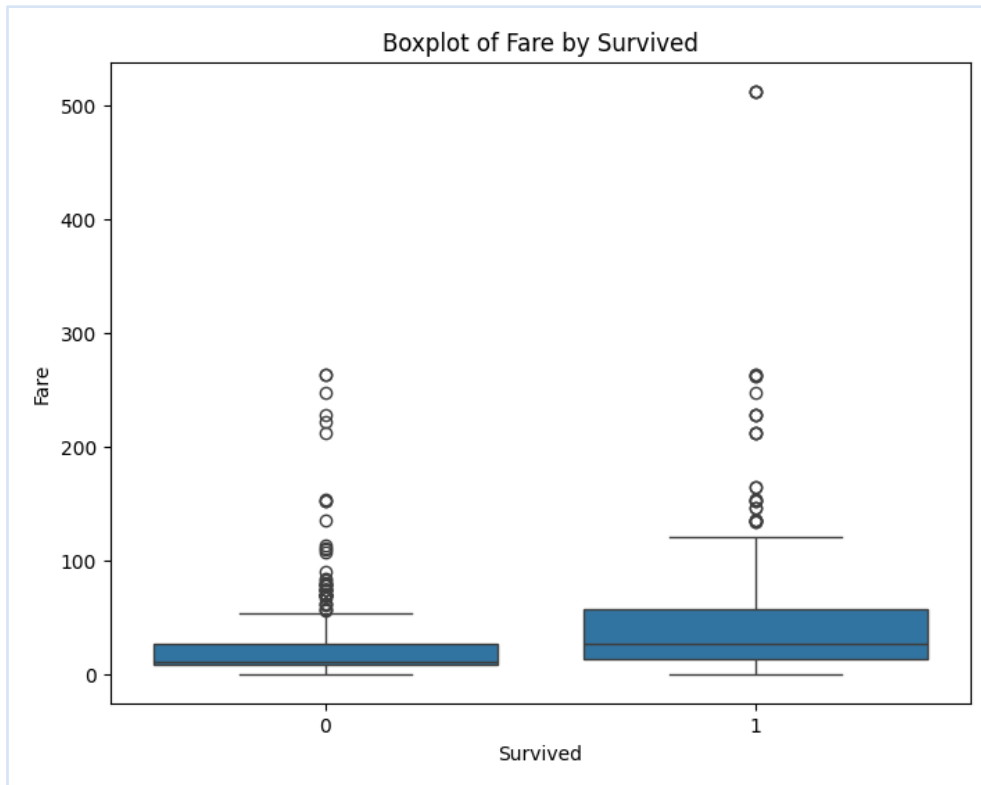
Relación entre variables (1/3)



Relación entre variables (2/3)



Relación entre variables (3/3)



Proporción de Survived=1 para cada valor de Sex:

Embarked

S 0.336957

C 0.553571

Q 0.389610

Name: count, dtype: float64

MODELOS SUPERVISADOS

CART

Classification and Regression Trees

Supervisado: árboles de decisión

Idea central

Los árboles de decisión dividen el espacio de los datos mediante una secuencia de preguntas binarias, llegando a una predicción interpretable paso a paso.

Árboles de decisión · el problema

Decidir si esperar por una mesa en un restaurante, según los siguientes atributos:

Alternativa <i>¿Hay un restaurante alternativo cerca?</i>	Precio <i>Rango de precios (\$, \$\$, \$\$\$)</i>
Bar <i>¿Hay un área de bar cómodo para esperar?</i>	Lluvia <i>¿Está lloviendo afuera?</i>
Viernes <i>¿Es hoy viernes o sábado?</i>	Reserva <i>¿Hemos hecho una reserva?</i>
Hambriento <i>¿Tenemos hambre?</i>	Tipo <i>Francés, italiano, tailandés o hamburguesa</i>
Personas <i>Número de personas (Ninguno, Algunos, Completo)</i>	Espera <i>Tiempo estimado (0-10, 10-30, 30-60, >60)</i>

Árboles de decisión • observaciones

#	Alt	Bar	Vie	Ham	Personas	Precio	Lluvia	Res	Tipo	Tiempo	Espera
1	V	F	F	V	Algunas	\$\$\$	F	V	Francés	0-10	V
2	V	F	F	V	Lleno	\$	F	F	Thai	30-60	F
3	F	V	F	F	Algunas	\$	F	F	Burger	0-10	V
4	V	F	V	V	Lleno	\$	F	F	Thai	10-30	V
5	V	F	V	F	Lleno	\$\$\$	F	V	Francés	>60	F
6	F	V	F	V	Algunas	\$\$	V	V	Italiano	0-10	V
7	F	V	F	F	Ninguna	\$	V	F	Burger	0-10	F
8	F	F	F	V	Algunas	\$\$	V	V	Thai	0-10	V
9	F	V	V	F	Lleno	\$	V	F	Burger	>60	F
10	V	V	V	V	Lleno	\$\$\$	F	V	Italiano	10-30	F
11	F	F	F	F	Ninguna	\$	F	F	Thai	0-10	F
12	V	V	V	V	Lleno	\$	F	F	Burger	30-60	V

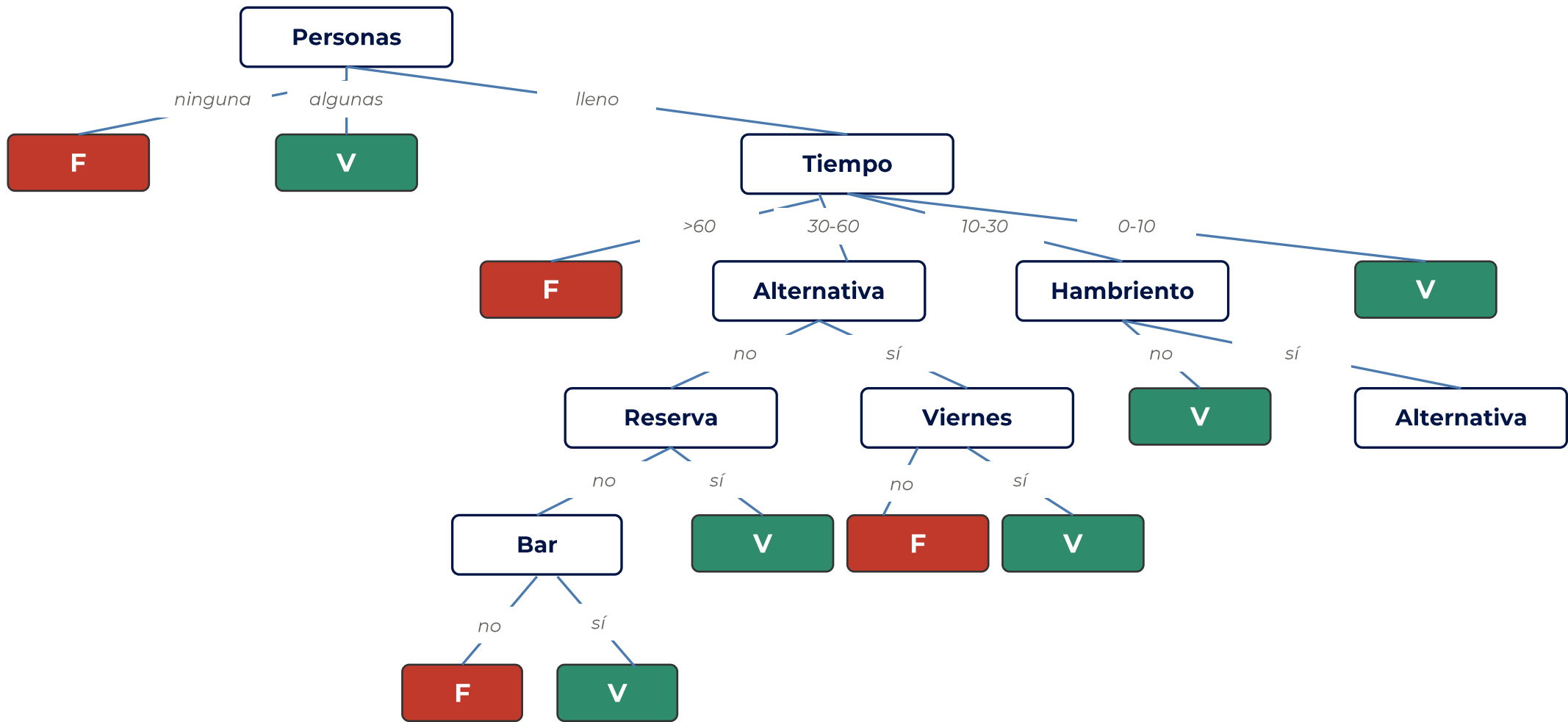
Se observó el comportamiento y las características de contexto de un grupo de personas que decidieron o no esperar.

Árboles de decisión • valores posibles

#	Alt	Bar	Vie	Ham	Personas	Precio	Lluvia	Res	Tipo	Tiempo	Espera
1	V	F	F	V	Algunas	\$\$\$	F	V	Francés	0-10	V
2	V	F	F	V	Lleno	\$	F	F	Thai	30-60	F
3	F	V	F	F	Algunas	\$	F	F	Burger	0-10	V
4	V	F	V	V	Lleno	\$	F	F	Thai	10-30	V
5	V	F	V	F	Lleno	\$\$\$	F	V	Francés	>60	F
6	F	V	F	V	Algunas	\$\$	V	V	Italiano	0-10	V
7	F	V	F	F	Ninguna	\$	V	F	Burger	0-10	F
8	F	F	F	V	Algunas	\$\$	V	V	Thai	0-10	V
9	F	V	V	F	Lleno	\$	V	F	Burger	>60	F
10	V	V	V	V	Lleno	\$\$\$	F	V	Italiano	10-30	F
11	F	F	F	F	Ninguna	\$	F	F	Thai	0-10	F
12	V	V	V	V	Lleno	\$	F	F	Burger	30-60	V

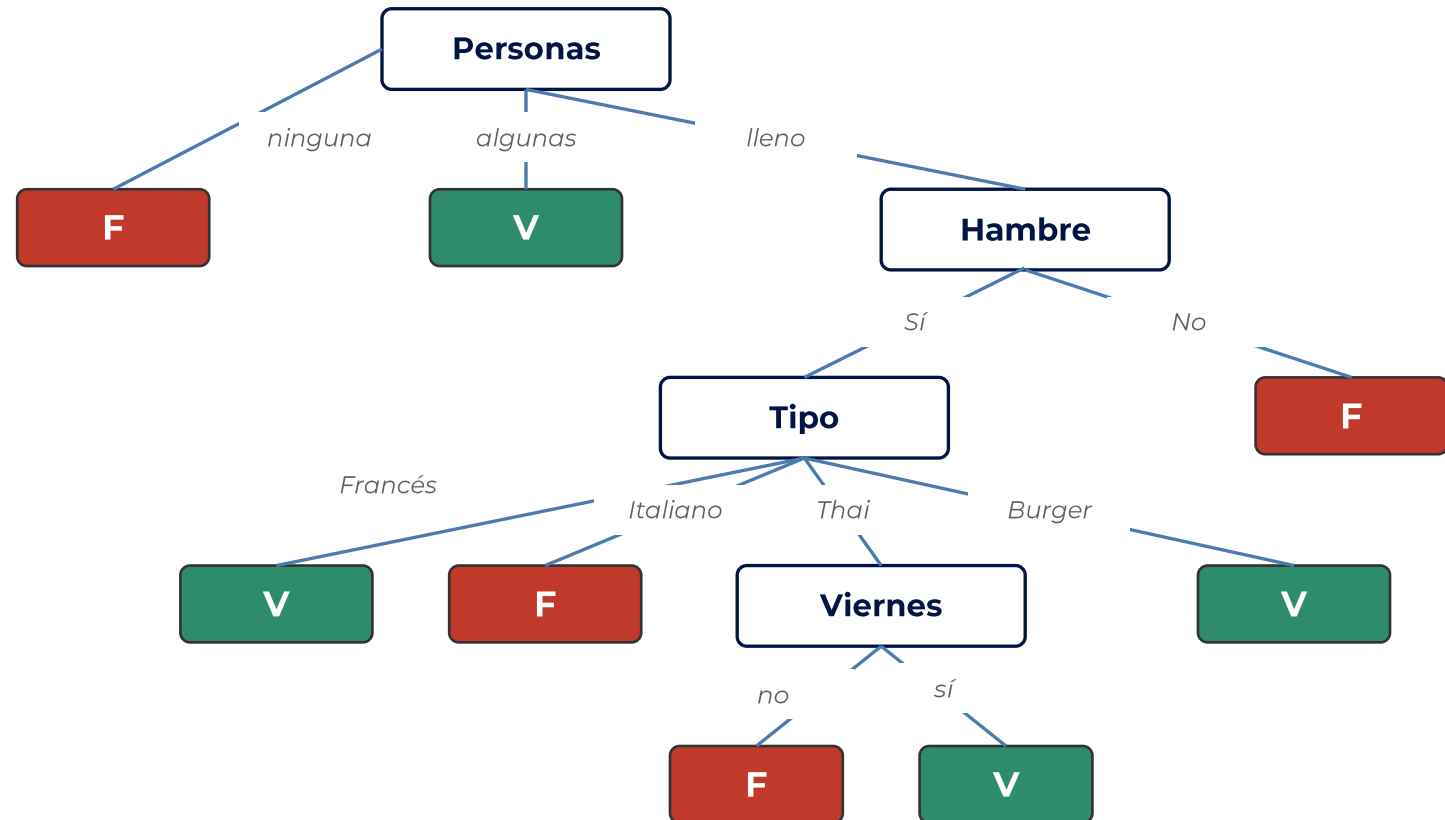
Tipo: Francés · Italiano · Thai · Burger Personas: Ninguna · Algunas · Lleno

Árboles de decisión · árbol completo



Árboles de decisión • árbol simplificado

- Es un árbol de decisión más simple que el original.
- No incluye preguntas sobre lluvia ni reserva porque se pueden clasificar todas las muestras sin ellas.
- Comete algunos errores, pero generaliza mejor.



Índice de Gini

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

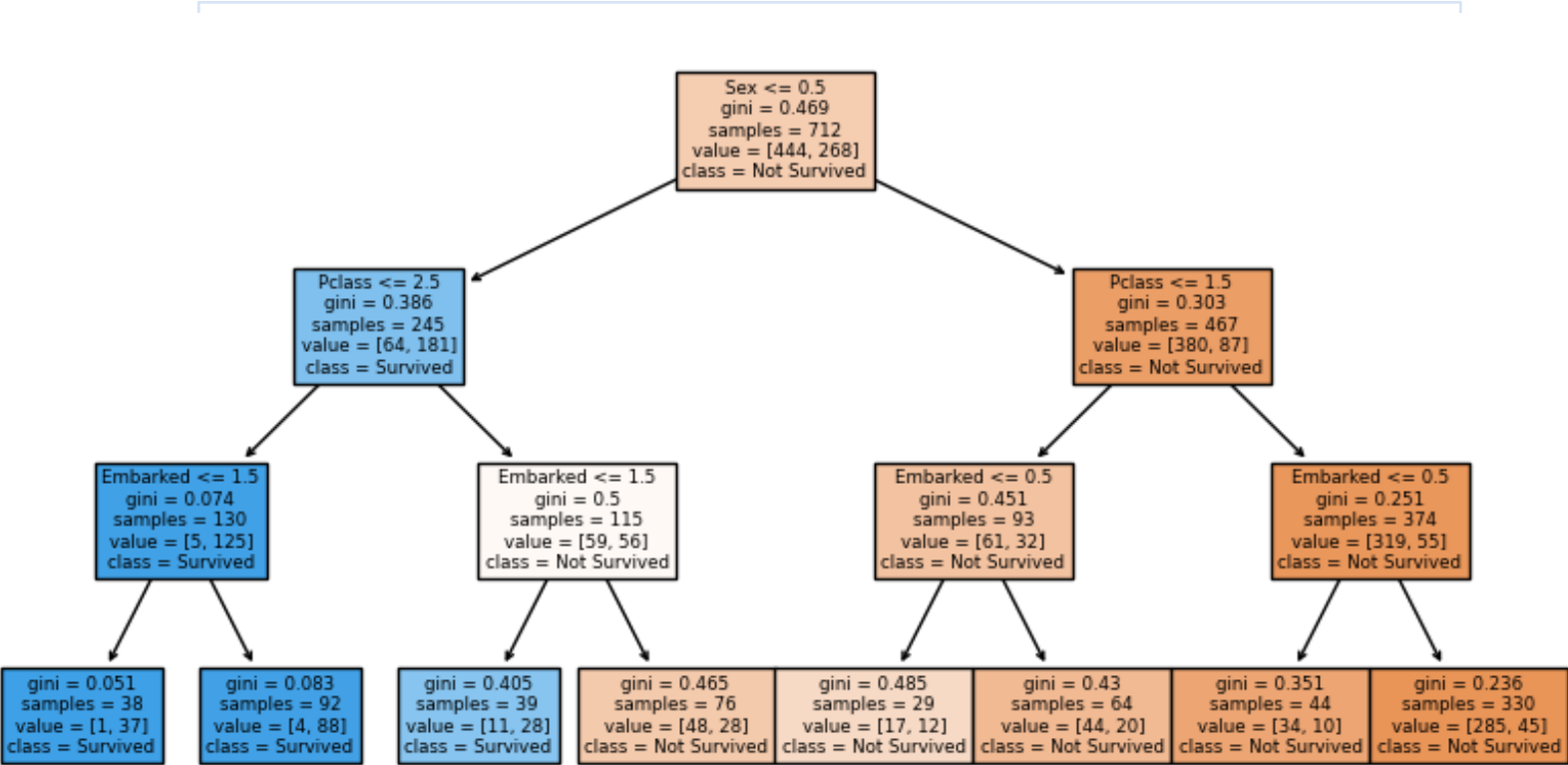
¿Qué mide?

Se utiliza en árboles de decisión para medir la impureza de un nodo, es decir, qué tan mezcladas están las clases.

- *Ejemplo: 60% de clientes que no abandonan y 40% que abandonan.*
- **Cuanto menor el Gini, más puro (homogéneo) es el nodo.**
- *Un Gini de 0 significa que todas las observaciones pertenecen a una sola clase.*

Importancia de las variables

Los árboles de decisión permiten entender fácilmente cuánto peso tiene cada una de las variables en la solución del problema.



¿Qué es CART?

Definición

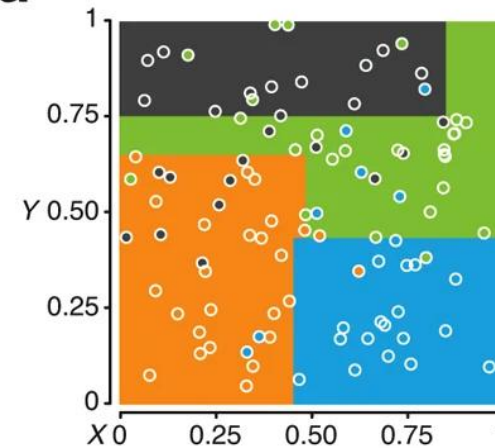
Método jerárquico divisivo propuesto por Breiman, Friedman, Olshen y Stone (1984). Se construye un árbol dividiendo grupos a partir de condiciones binarias.

Referencias del ejemplo

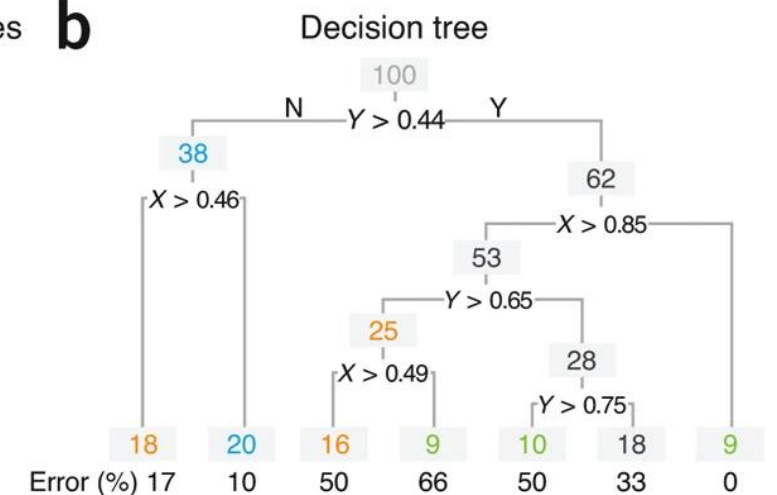
Izq = sí · Sex: Mujer=0, Varón=1

EAnálítica de Datosrked: C=0, Q=1, S=2, NULL=3

a Partitioning of two predictor variables

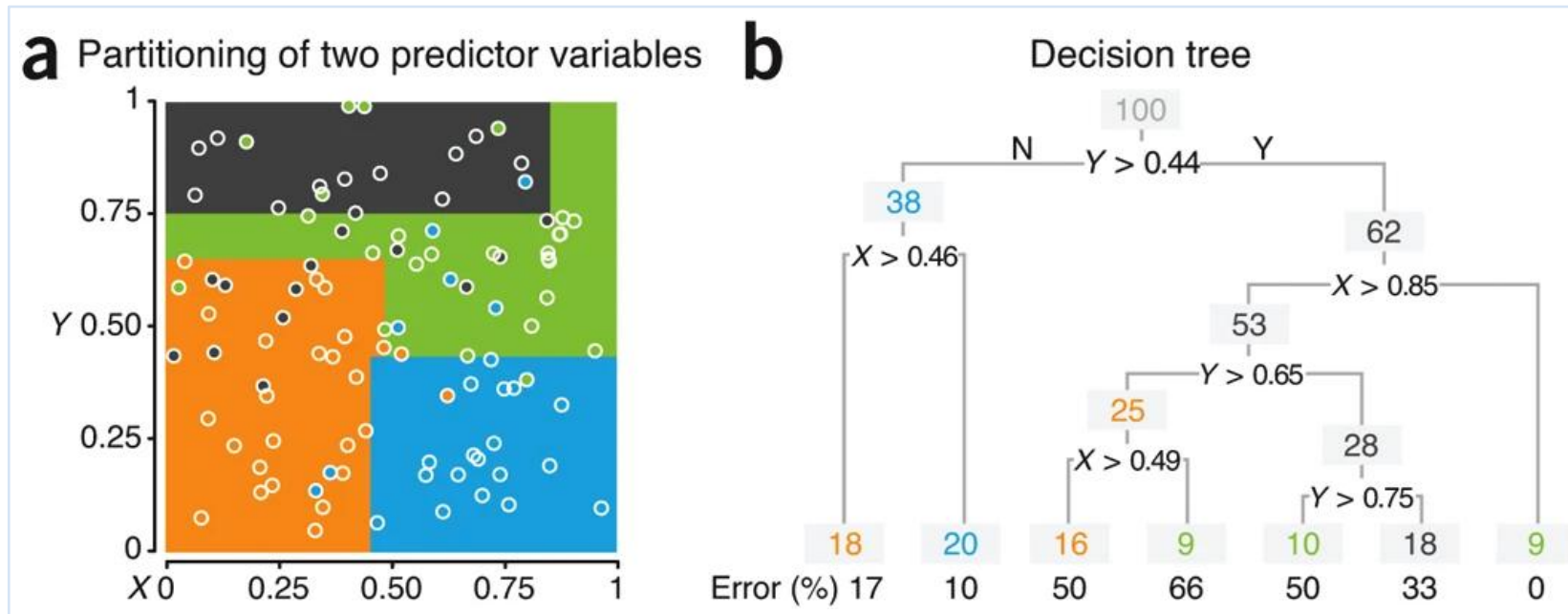


b



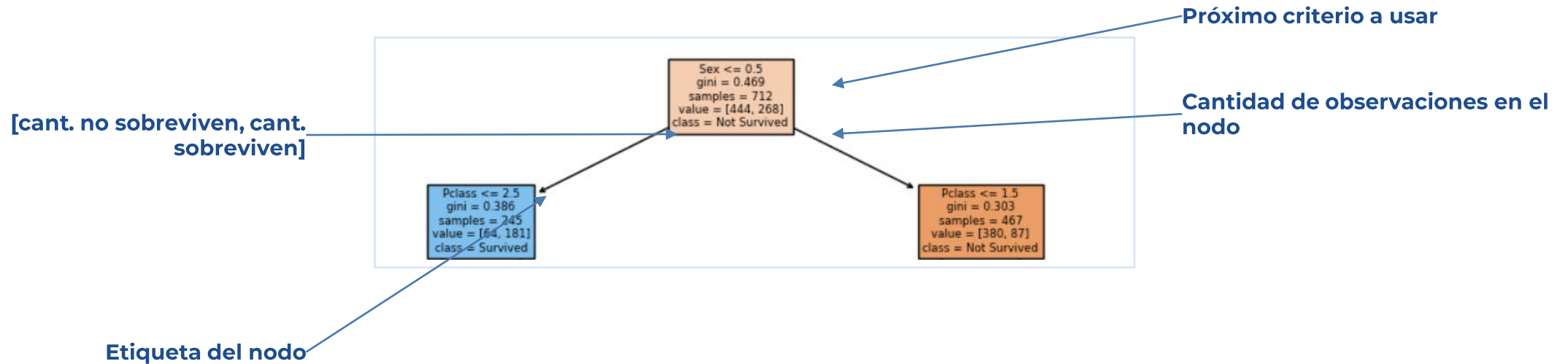
Particiones del espacio

Los árboles determinan particiones del espacio paralelas a los ejes.

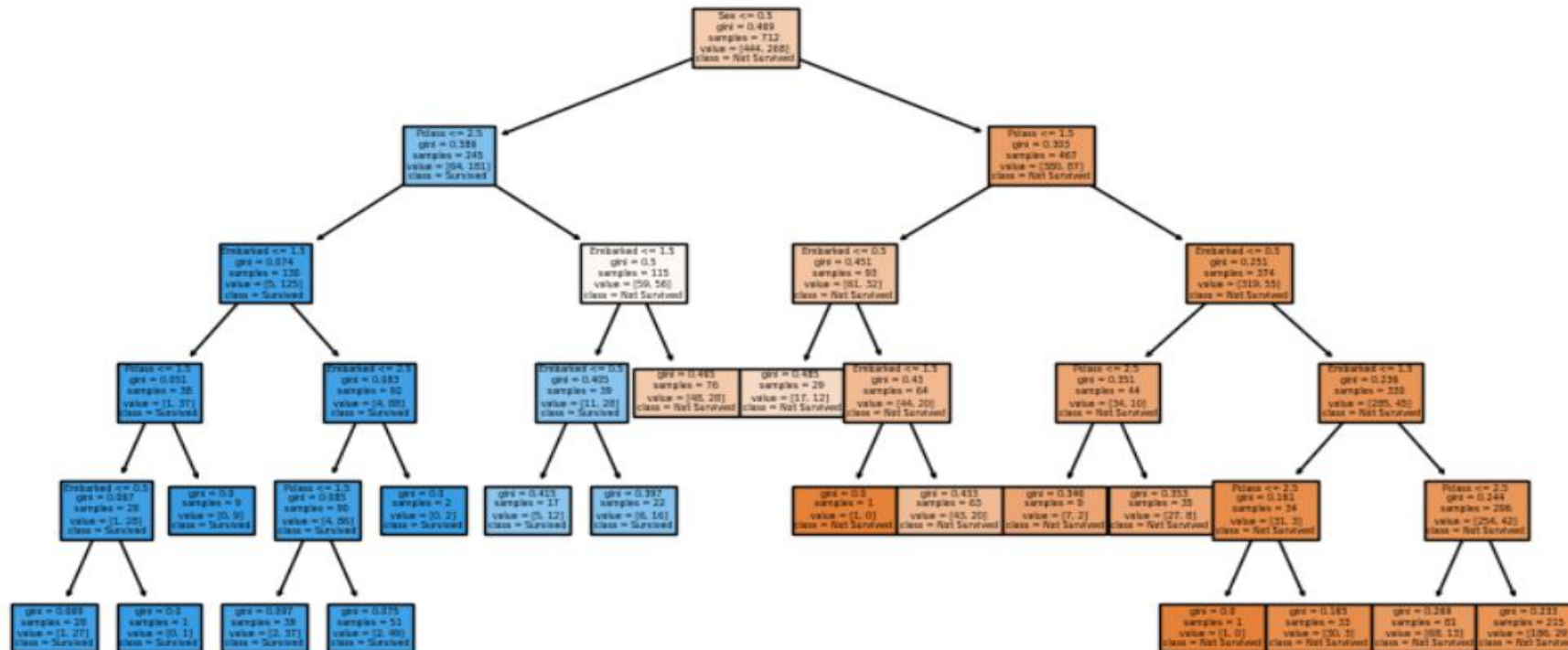


Crecimiento del árbol

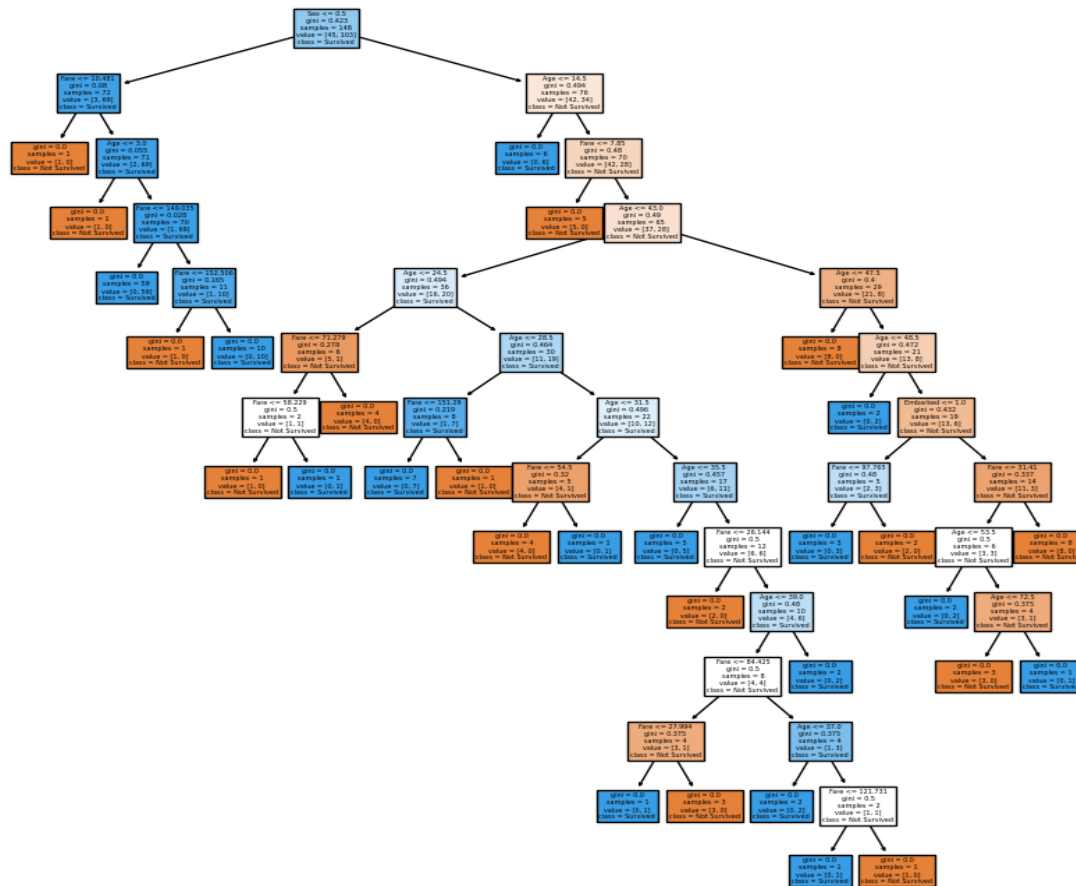
1. Al comienzo tenemos todas las observaciones en el primer nodo (712 en nuestro ejemplo). El algoritmo recorre todas las particiones posibles y elige la mejor: la que más mejora la pureza de los nodos generados. A cada nodo se le asigna la etiqueta mayoritaria.



2. Repetir el paso anterior hasta obtener el árbol maximal. Se pueden definir criterios de corte: número mínimo de observaciones en un nodo, altura máxima del árbol.



Crecimiento · si usáramos más variables...



Sin interpretación
y genera OVERFITTING

Pruning (poda)

Para evitar el sobreajuste debemos podar el árbol, minimizando una función como la siguiente:

$$R_{\alpha}(T) = \sum_{i=1}^{|T|} \text{criterio de impureza} + \alpha |T|$$

Donde $|T|$ es la cantidad de hojas del árbol T y α es un hiperparámetro que se debe elegir.

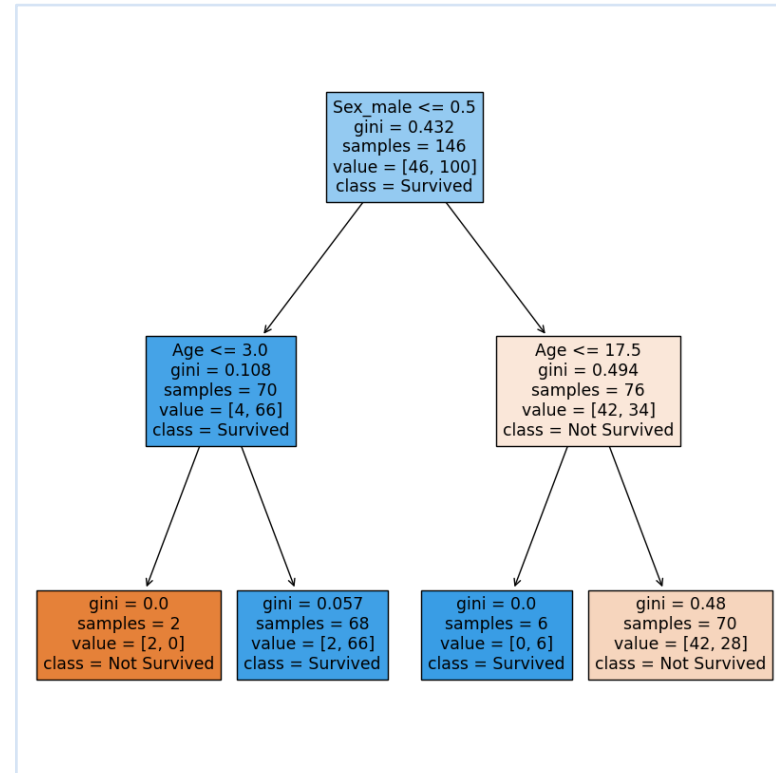
El criterio de impureza para podar es muy similar al criterio que se utiliza para elegir 'la mejor' partición al hacer crecer el árbol.

Pruning · ejemplo con $\alpha = 0.02$

Reporte de clasificación:

	precision	recall	f1-score	support
0	0.71	0.45	0.56	11
1	0.80	0.92	0.86	26
accuracy			0.78	37
macro avg	0.76	0.69	0.71	37
weighted avg	0.77	0.78	0.77	37

```
tree_model = DecisionTreeClassifier(
    ccp_alpha=0.02)
```



Criterios de impureza

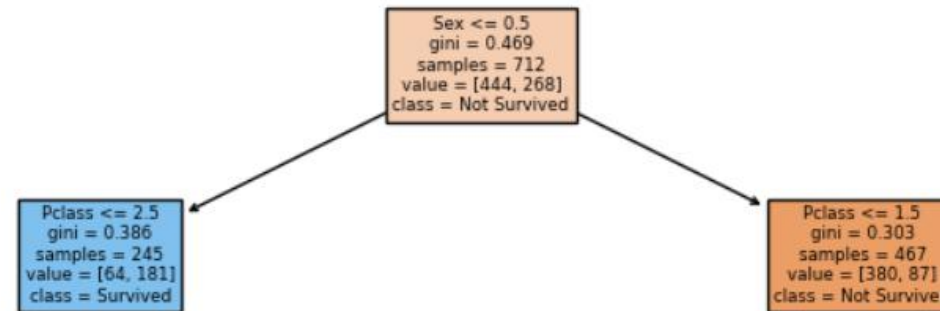
Son los criterios que se usan tanto para decidir 'la mejor' partición al hacer crecer el árbol como para minimizar la función de pérdida al podar. Indican qué tan homogéneos son los nodos.

Error de clasificación

Índice de Gini

Entropía

Criterios de impureza • ejemplo Gini



$$0,62 (1-0,62) + 0,37 (1-0,37)$$

$$0,26 (1-0,26) + 0,74 (1-0,74)$$

$$0,81 (1-0,81) + 0,19 (1-0,19)$$

Gini • impureza media

Supongamos que tenemos un nodo con las siguientes observaciones:

Clase	Observaciones
Sobrevive (clase a)	40
No sobrevive (clase b)	60

1. Cálculo de las proporciones

- $p_A = \frac{40}{100} = 0.4$
- $p_B = \frac{60}{100} = 0.6$

2. Índice de Gini para este nodo

$$\begin{aligned} & 0,4 (1-0,4) + 0,6 (1-0,6) \\ &= 0,4 \cdot 0,6 + 0,6 \cdot 0,4 \\ &= 0,24 + 0,24 \\ &= 0,48 \end{aligned}$$

Interpretación

Un índice de Gini de 0,48 indica que el nodo contiene cierto nivel de impureza (0 sería pureza total, todos los elementos de la misma clase). Si un nodo tiene un Gini bajo, la mayoría de las observaciones pertenecen a una misma clase, lo cual es deseable en un árbol de decisión.

Gini • ejemplo de pureza total

Supongamos que tenemos un nodo con las siguientes observaciones:

Clase	Observaciones
Sobrevive (clase a)	50
No sobrevive (clase b)	0

1. Cálculo de las proporciones

$$p_{\text{Sobrevive}} = \frac{50}{50} = 1 \quad \text{y} \quad p_{\text{No Sobrevive}} = \frac{0}{50} = 0$$

2. Índice de Gini para este nodo

$$\begin{aligned} &1 (1-1) + 0 (1-0) \\ &= 0 + 0 \\ &= 0 \end{aligned}$$

Interpretación

Un índice de Gini de 0 se da cuando un nodo tiene pureza total, es decir, todas las observaciones del nodo corresponden a la misma clase (sobrevive o no sobrevive).

Gini • ejemplo de impureza total

Supongamos que tenemos un nodo con las siguientes observaciones:

Clase	Observaciones
Sobrevive (clase a)	25
No sobrevive (clase b)	25

1. Cálculo de las proporciones

$$p_{\text{Sobrevive}} = \frac{25}{50} = 0.5 \quad \text{y} \quad p_{\text{No Sobrevive}} = \frac{25}{50} = 0.5$$

2. Índice de Gini para este nodo

$$\begin{aligned} & 0,5 (1-0,5) + 0,5 (1-0,5) \\ &= 0,5 \cdot 0,5 + 0,5 \cdot 0,5 \\ &= 0,5 \end{aligned}$$

Interpretación

Un Gini de 0,5 se da cuando el nodo tiene impureza máxima: las clases están distribuidas de forma equilibrada. El nodo no es útil para clasificar, ya que ambas clases tienen la misma probabilidad. Por eso los algoritmos buscan dividir los datos minimizando la impureza en los nodos hijos.

Ventajas y desventajas

Ventajas

- *Fáciles de interpretar*
- *Adaptabilidad*
- *Visualización clara*

Desventajas

- *Inestabilidad · alta varianza*
- *Sobreajuste (overfitting)*

Hiperparámetros del modelo

max_depth *Profundidad máxima del árbol*

min_samples_split *Número mínimo de observaciones por nodo*

min_samples_leaf *Número mínimo de datos por hoja*

ccp_alpha *Alpha de poda por complejidad de costo*

```
tree_model = DecisionTreeClassifier(random_state=42, max_depth=None, min_samples_split=2, min_samples_leaf=1, ccp_alpha=0.02)
```

MODELOS SUPERVISADOS

Random Forest

Ensamblados de árboles para reducir el overfitting

¿Qué son?

Ensamble

Un ensamble (ensemble) combina múltiples modelos de machine learning para crear modelos más potentes.

- *Uno de los problemas de los árboles de decisión es que hacen overfitting rápidamente.*
- *Un random forest es una colección de árboles, cada uno levemente diferente a los otros.*
- *Promediando sus resultados se reduce el overfitting.*
- *Se inserta randomización para que cada árbol sea diferente.*

```
from sklearn.ensemble import \
    RandomForestClassifier

X_train, y_train = ...

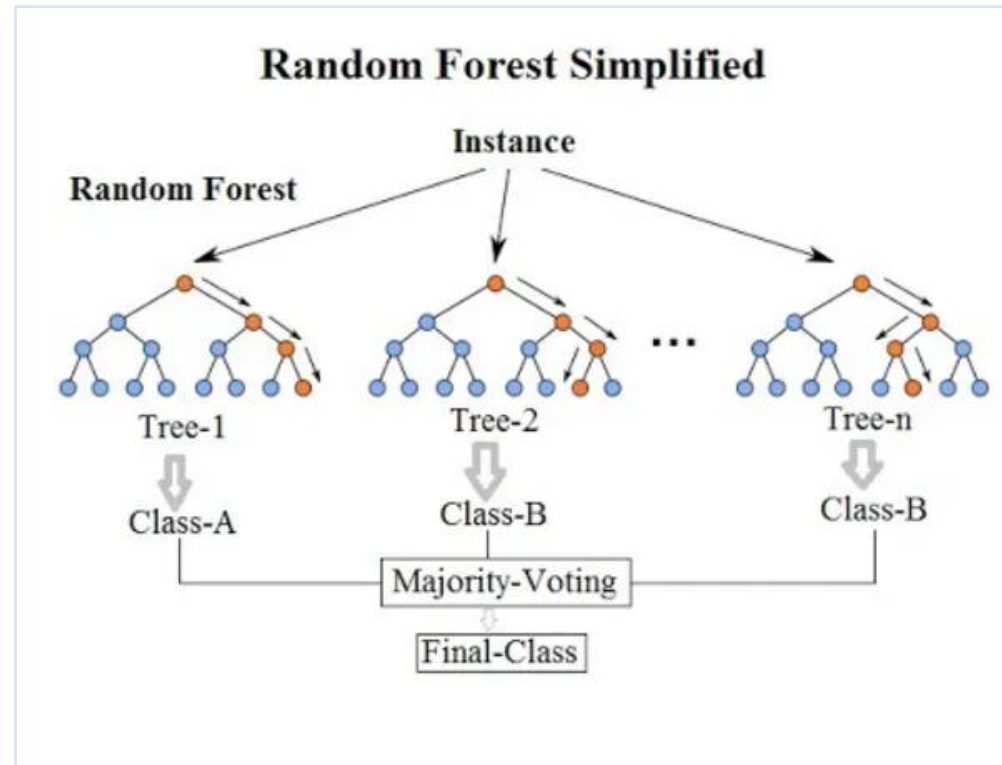
rfc = RandomForestClassifier()
rfc.fit(X_train, y_train)

prediction = rfc.predict(X_test)
```

Algoritmo

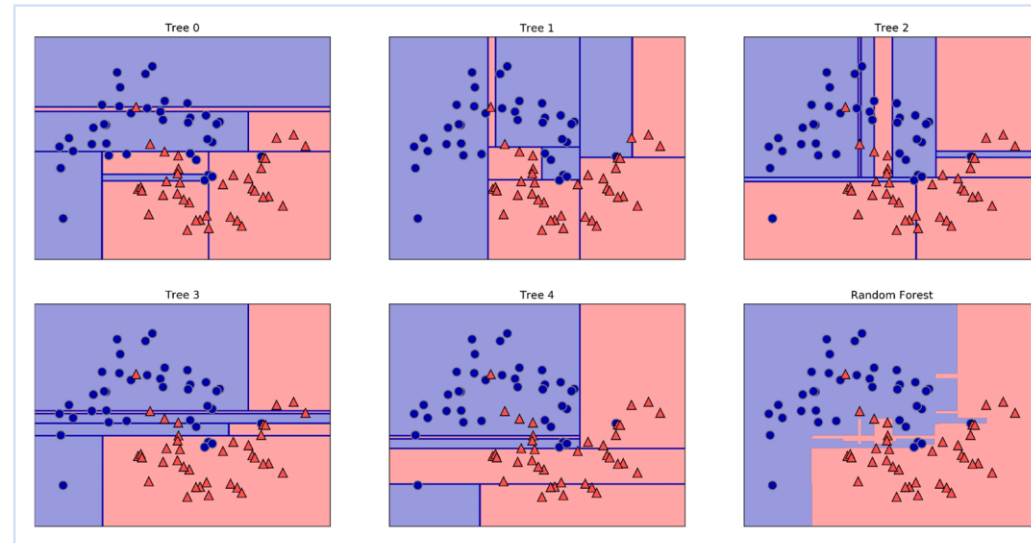
- **Hacer crecer B árboles:**
 - *Extraer una muestra bootstrap de tamaño n de la muestra de entrenamiento.*
 - *Hacer crecer un árbol con esa muestra; en cada nodo: elegir m variables entre todas las posibles, elegir la mejor alternativa de división y hacer la división binaria del nodo.*
- **Ensamblar los árboles obtenidos. Para predecir, asignar a la observación la clase más votada entre todos los árboles.**

Esquema conceptual



¿Selección de características o datos?

- Se observa que las fronteras de decisión de cada árbol son diferentes y cada uno comete errores.
- Como se usa bootstrapping, cada árbol se construye con un set de datos diferente.
- El overfitting es menor que el de cada árbol individual.



`n_estimators = 5`

Volviendo al ejemplo del Titanic

Configuración

Modelo con los mismos parámetros que veníamos usando, ensamblando 1000 árboles. Se dejó `max_features` por defecto: `sqrt(p)`.

Se pierde interpretabilidad pero mejora el poder predictivo.

Reporte de clasificación:

	precision	recall	f1-score	support
0	0.80	0.62	0.70	13
1	0.81	0.92	0.86	24
accuracy			0.81	37
macro avg	0.81	0.77	0.78	37
weighted avg	0.81	0.81	0.80	37

```
tree_model.feature_importances_
```

F1-Score: uniendo precisión y recall

Métrica para evaluar modelos de clasificación, especialmente con clases desbalanceadas. Combina la precisión y la cobertura (recall) en un solo valor.

Precisión (Precision)

Qué proporción de las predicciones positivas fueron correctas.

Valor cercano a 1

Alta precisión y alta cobertura: buen desempeño del modelo.

Cobertura (Recall)

Qué proporción de los positivos reales fueron efectivamente identificados.

Valor cercano a 0

No predice bien las clases positivas o tiene muchos falsos positivos.

Checkpoint: el proceso en sklearn

En sklearn, el proceso de creación de modelos es muy simple y similar entre algoritmos:

```
from sklearn.tree \
    import DecisionTreeClassifier

X_train, y_train = ...

dtc = DecisionTreeClassifier()
dtc.fit(X_train, y_train)

prediction = \
    dtc.predict(X_test)
```

```
from sklearn.ensemble \
    import RandomForestClassifier

X_train, y_train = ...

rfc = RandomForestClassifier()
rfc.fit(X_train, y_train)

prediction = \
    rfc.predict(X_test)
```

Patrón general

```
import model
split_data()
model(...)
model.fit(...)
model.predict()
```

Hiperparámetros del modelo

max_depth

Profundidad máxima del árbol

ccp_alpha

Alpha de poda por complejidad de costo

min_samples_split

Número mínimo de observaciones por nodo

max_features

Cantidad máxima de variables por partición

min_samples_leaf

Número mínimo de datos por hoja

n_estimators

Cantidad de árboles del ensamble

```
tree_model = RandomForestClassifier(random_state=42, max_depth=None, min_samples_split=2, min_samples_leaf=1, ccp_alpha=0.02, max_features=3, n_estimators=10)
```

MACHINE LEARNING

Grid Search y Random Search

Evaluación robusta de la performance de generalización

Cross validation

- *Cross validation es un método estadístico de evaluación de la performance de la generalización, más estable que usar un único split en training y test set.*
- *Los datos se particionan repetitivamente y se entrenan varios modelos.*
- *La versión más común es k-fold cross validation, donde k es un número definido por el usuario, generalmente 5 o 10.*

Training, validation y test

Se suele reservar el test set para una prueba final del modelo, por lo que para pruebas parciales se genera un validation set.

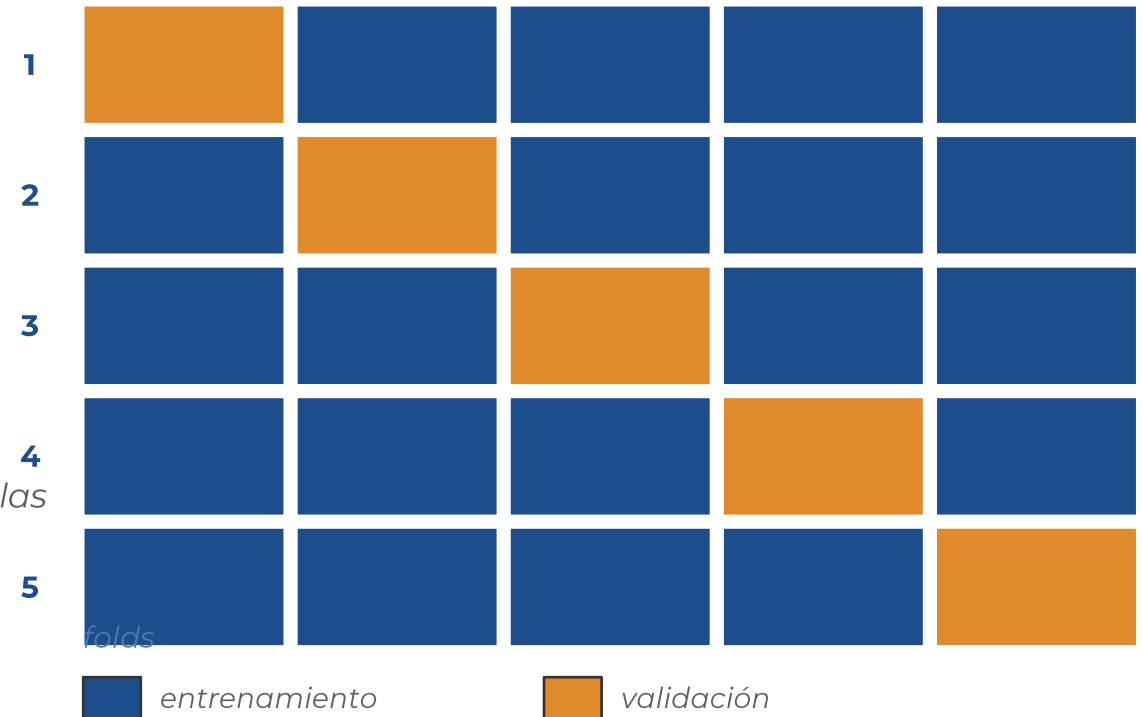


- **Training set:** lo usamos para construir el modelo.
- **Validation set:** lo usamos para ajustar los modelos.
- **Test set:** lo usamos para la evaluación final de la performance.

k-fold cross validation

En 5-fold cross validation los datos se parten en 5 folds. En cada iteración un fold distinto actúa como validación y el resto como entrenamiento.

En k-fold cross-validation las particiones no son adyacentes, sino aleatorias y, a veces, estratificadas para mantener la distribución de las clases.



Cross validation • ventajas y desventajas

Ventajas

- *Cada muestra está en el set de prueba exactamente una vez: el modelo debe generalizar bien sobre todo el dataset.*
- *Tener múltiples splits informa sobre la sensibilidad del modelo a la selección del training set.*
- *Se usan los datos más eficientemente: con 5 folds se entrena con el 80% y con 10 folds con el 90%.*

Desventajas

- *Costo computacional: se entrenan k modelos en lugar de uno, por lo que el proceso es k veces más lento que un split simple.*

Grid Search

- *Técnica de optimización de hiperparámetros que busca la combinación óptima que maximiza el rendimiento del modelo.*
- *Se define una cuadrícula (grid) de valores posibles y se prueban todas las combinaciones de manera exhaustiva.*
- *Cada combinación se entrena y valida con cross-validation.*
- *Garantiza explorar todo el espacio, pero puede ser computacionalmente costoso con muchos parámetros.*

```
model = RandomForestClassifier()
param_grid = {
    'n_estimators': [100,200,500,1000],
    'max_depth': [None,10,20,30,40,50],
    'min_samples_split': [2,5,10],
    'min_samples_leaf': [1,2,4]
}
grid_search = GridSearchCV(
    estimator=model,
    param_grid=param_grid, cv=5)
grid_search.fit(X_train, y_train)
best = grid_search.best_params_
```

Random Search

- *Selecciona combinaciones aleatorias de hiperparámetros dentro de un espacio definido.*
- *A diferencia de Grid Search, prueba un número limitado de combinaciones aleatorias: es más eficiente computacionalmente.*
- *Cada combinación se entrena y valida con validación cruzada.*
- *Útil cuando el espacio es grande; al aumentar las iteraciones crece la probabilidad de encontrar combinaciones óptimas.*

```
model = RandomForestClassifier()
param_dist = {
    'n_estimators': [100,200,500,1000],
    'max_depth': [None,10,20,30,40,50],
    'min_samples_split': [2,5,10],
    'min_samples_leaf': [1,2,4],
    'bootstrap': [True, False]
}
random_search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_dist,
    n_iter=10, cv=5, random_state=42)
random_search.fit(X_train, y_train)
```

CIERRE

¡Gracias!

Árboles de decisión, Random Forest y validación cruzada — Analítica de Datos

Profesor Ing. Pablo Sciolla

Universidad de San Andrés — Analítica de Datos